



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 26 de febrero del 2026

CALIFICACIÓN: _____

ACTIVIDADES

Letras de mi nombre

Como primera actividad debíamos dibujar las iniciales de nuestros nombres en diagonal de abajo hacia arriba, cada letra de un color diferente. Para ello reutilizamos las coordenadas que ya habíamos fijado de nuestras iniciales en la práctica 1.

Dentro de la función `void CrearLetrasyFiguras()`, reutilizamos los vértices para la letra E, la letra C y la letra M. También les asignamos colores de tonos rosas/rojizos distintos para cada letra. Además, por cada letra utilizamos `MeshColor` para reservar memoria dinámica, creando así el objeto de la misma clase, nombrándolo como `LetraE/C/M`.

Letra E:

```
186 // -----
187 // LETRAS [7,8,9] -----
188 GLfloat vertices_LetraE[] = {
189     // Rectangulo vertical
190     -0.6f,  0.1f,  0.0f,      0.251f, 0.020f, 0.078f, //T1
191     -0.6f, -0.4f,  0.0f,      0.251f, 0.020f, 0.078f,
192     -0.5f, -0.4f,  0.0f,      0.251f, 0.020f, 0.078f,
193     -0.6f,  0.1f,  0.0f,      0.251f, 0.020f, 0.078f, //T2
194     -0.5f, -0.4f,  0.0f,      0.251f, 0.020f, 0.078f,
195     -0.5f,  0.1f,  0.0f,      0.251f, 0.020f, 0.078f,
196     // Rectangulo superior
197     -0.5f,  0.1f,  0.0f,      0.251f, 0.020f, 0.078f, //T3
198     -0.5f,  0.0f,  0.0f,      0.251f, 0.020f, 0.078f,
199     -0.3f,  0.0f,  0.0f,      0.251f, 0.020f, 0.078f,
200     -0.5f,  0.1f,  0.0f,      0.251f, 0.020f, 0.078f, //T4
201     -0.3f,  0.0f,  0.0f,      0.251f, 0.020f, 0.078f,
202     -0.3f,  0.1f,  0.0f,      0.251f, 0.020f, 0.078f,
203     // Rectangulo medio
204     -0.5f, -0.1f,  0.0f,      0.251f, 0.020f, 0.078f, //T5
205     -0.5f, -0.2f,  0.0f,      0.251f, 0.020f, 0.078f,
206     -0.4f, -0.2f,  0.0f,      0.251f, 0.020f, 0.078f,
207     -0.5f, -0.1f,  0.0f,      0.251f, 0.020f, 0.078f, //T6
208     -0.4f, -0.2f,  0.0f,      0.251f, 0.020f, 0.078f,
209     -0.4f, -0.1f,  0.0f,      0.251f, 0.020f, 0.078f,
210     // Rectangulo inferior
211     -0.5f, -0.3f,  0.0f,      0.251f, 0.020f, 0.078f, //T7
212     -0.5f, -0.4f,  0.0f,      0.251f, 0.020f, 0.078f,
213     -0.3f, -0.4f,  0.0f,      0.251f, 0.020f, 0.078f,
214     -0.5f, -0.3f,  0.0f,      0.251f, 0.020f, 0.078f, //T8
215     -0.3f, -0.4f,  0.0f,      0.251f, 0.020f, 0.078f,
216     -0.3f, -0.3f,  0.0f,      0.251f, 0.020f, 0.078f
217 };
218 MeshColor* LetraE = new MeshColor();
219 LetraE->CreateMeshColor(vertices_LetraE, 144); // 24*6=144
220 meshColorList.push_back(LetraE);
```

Letra C:

```
222 GLfloat vertices_LetraC[] = {
223     // Rectangulo vertical
224     -0.2f, 0.3f, 0.0f, 0.529f, 0.118f, 0.220f, //T1
225     -0.2f, -0.2f, 0.0f, 0.529f, 0.118f, 0.220f,
226     -0.1f, -0.2f, 0.0f, 0.529f, 0.118f, 0.220f,
227     -0.2f, 0.3f, 0.0f, 0.529f, 0.118f, 0.220f, //T2
228     -0.1f, -0.2f, 0.0f, 0.529f, 0.118f, 0.220f,
229     -0.1f, 0.3f, 0.0f, 0.529f, 0.118f, 0.220f,
230     // Rectangulo superior
231     -0.1f, 0.3f, 0.0f, 0.529f, 0.118f, 0.220f, //T3
232     -0.1f, 0.2f, 0.0f, 0.529f, 0.118f, 0.220f,
233     0.1f, 0.2f, 0.0f, 0.529f, 0.118f, 0.220f,
234     -0.1f, 0.3f, 0.0f, 0.529f, 0.118f, 0.220f, //T4
235     0.1f, 0.2f, 0.0f, 0.529f, 0.118f, 0.220f,
236     0.1f, 0.3f, 0.0f, 0.529f, 0.118f, 0.220f,
237     // Rectangulo inferior
238     -0.1f, -0.1f, 0.0f, 0.529f, 0.118f, 0.220f, //T5
239     -0.1f, -0.2f, 0.0f, 0.529f, 0.118f, 0.220f,
240     0.1f, -0.2f, 0.0f, 0.529f, 0.118f, 0.220f,
241     -0.1f, -0.1f, 0.0f, 0.529f, 0.118f, 0.220f, //T6
242     0.1f, -0.2f, 0.0f, 0.529f, 0.118f, 0.220f,
243     0.1f, -0.1f, 0.0f, 0.529f, 0.118f, 0.220f
244 };
245 MeshColor* LetraC = new MeshColor();
246 LetraC->CreateMeshColor(vertices_LetraC, 108); // 18*6
247 meshColorList.push_back(LetraC);
```

Letra M:

```
249 GLfloat vertices_LetraM[] = {
250     //Rectangulo vertical izquierdo
251     0.2f, 0.5f, 0.0f, 0.749f, 0.361f, 0.459f, //T1
252     0.2f, 0.0f, 0.0f, 0.749f, 0.361f, 0.459f,
253     0.3f, 0.0f, 0.0f, 0.749f, 0.361f, 0.459f,
254     0.2f, 0.5f, 0.0f, 0.749f, 0.361f, 0.459f, //T2
255     0.3f, 0.0f, 0.0f, 0.749f, 0.361f, 0.459f,
256     0.3f, 0.5f, 0.0f, 0.749f, 0.361f, 0.459f,
257     //Rectangulo vertical derecho
258     0.5f, 0.5f, 0.0f, 0.749f, 0.361f, 0.459f, //T3
259     0.5f, 0.0f, 0.0f, 0.749f, 0.361f, 0.459f,
260     0.6f, 0.0f, 0.0f, 0.749f, 0.361f, 0.459f,
261     0.5f, 0.5f, 0.0f, 0.749f, 0.361f, 0.459f, //T4
262     0.6f, 0.0f, 0.0f, 0.749f, 0.361f, 0.459f,
263     0.6f, 0.5f, 0.0f, 0.749f, 0.361f, 0.459f,
264     // Diagonales
265     0.3f, 0.5f, 0.0f, 0.749f, 0.361f, 0.459f, //T5
266     0.4f, 0.4f, 0.0f, 0.749f, 0.361f, 0.459f,
267     0.3f, 0.3f, 0.0f, 0.749f, 0.361f, 0.459f,
268     0.3f, 0.3f, 0.0f, 0.749f, 0.361f, 0.459f, //T6
269     0.4f, 0.4f, 0.0f, 0.749f, 0.361f, 0.459f,
270     0.4f, 0.2f, 0.0f, 0.749f, 0.361f, 0.459f,
271     0.5f, 0.5f, 0.0f, 0.749f, 0.361f, 0.459f, //T7
272     0.4f, 0.4f, 0.0f, 0.749f, 0.361f, 0.459f,
273     0.5f, 0.3f, 0.0f, 0.749f, 0.361f, 0.459f,
274     0.5f, 0.3f, 0.0f, 0.749f, 0.361f, 0.459f, //T8
275     0.4f, 0.4f, 0.0f, 0.749f, 0.361f, 0.459f,
276     0.4f, 0.2f, 0.0f, 0.749f, 0.361f, 0.459f
277 };
278 MeshColor* LetraM = new MeshColor();
279 LetraM->CreateMeshColor(vertices_LetraM, 144); // 24*6
280 meshColorList.push_back(LetraM);
```

Como consideraciones colocamos una proyección ortogonal descomentando la instrucción `glm::mat4 projection = glm::ortho(-1.0f, 1.0f, -1.0f, 1.0f, 0.1f, 100.0f);` y comentando la proyección en perspectiva, además nos aseguramos de usar el segundo set de shaders para las letras.

```
345     shaderList[1].useShader(); //segundo set de shaders para las letras
346     uniformModel = shaderList[1].getModelLocation();
347     uniformProjection = shaderList[1].getProjectLocation();
```

Finalmente, nuevamente dentro de nuestro ciclo while que permanece mientras no cerremos la ventana, para cada letra realizamos el posicionamiento y renderizado independiente de cada modelo mediante transformaciones geométricas, específicamente mediante la traslación para exagerar un poco más que las letras estén en diagonal de abajo hacia arriba.

```
350     // DIBUJO DE LETRAS -----
351     // -----
352     // --- Letra E
353     model = glm::mat4(1.0);
354     model = glm::translate(model, glm::vec3(-0.3f, -0.3f, -1.2f));
355     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
356     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
357     meshColorList[7]->RenderMeshColor();
358     // --- Letra C-
359     model = glm::mat4(1.0);
360     model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.2f));
361     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
362     meshColorList[8]->RenderMeshColor();
363     // --- Letra M
364     model = glm::mat4(1.0);
365     model = glm::translate(model, glm::vec3(0.3f, 0.3f, -1.2f));
366     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
367     meshColorList[9]->RenderMeshColor();
```

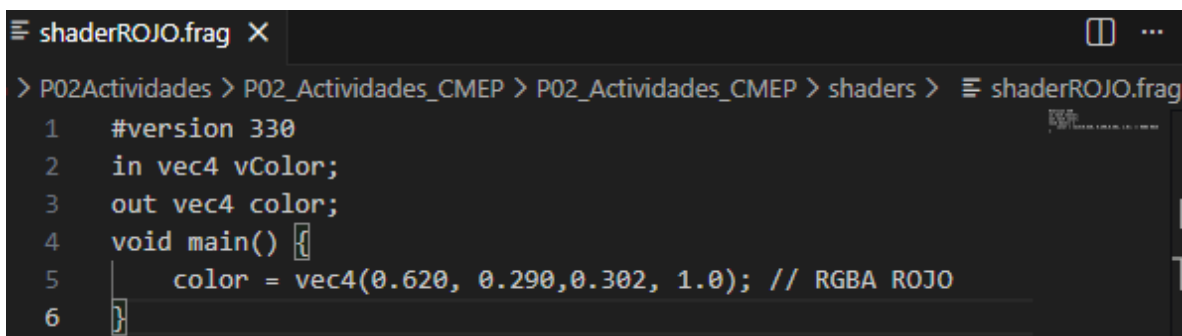
Ejecución



Dibujo de casa

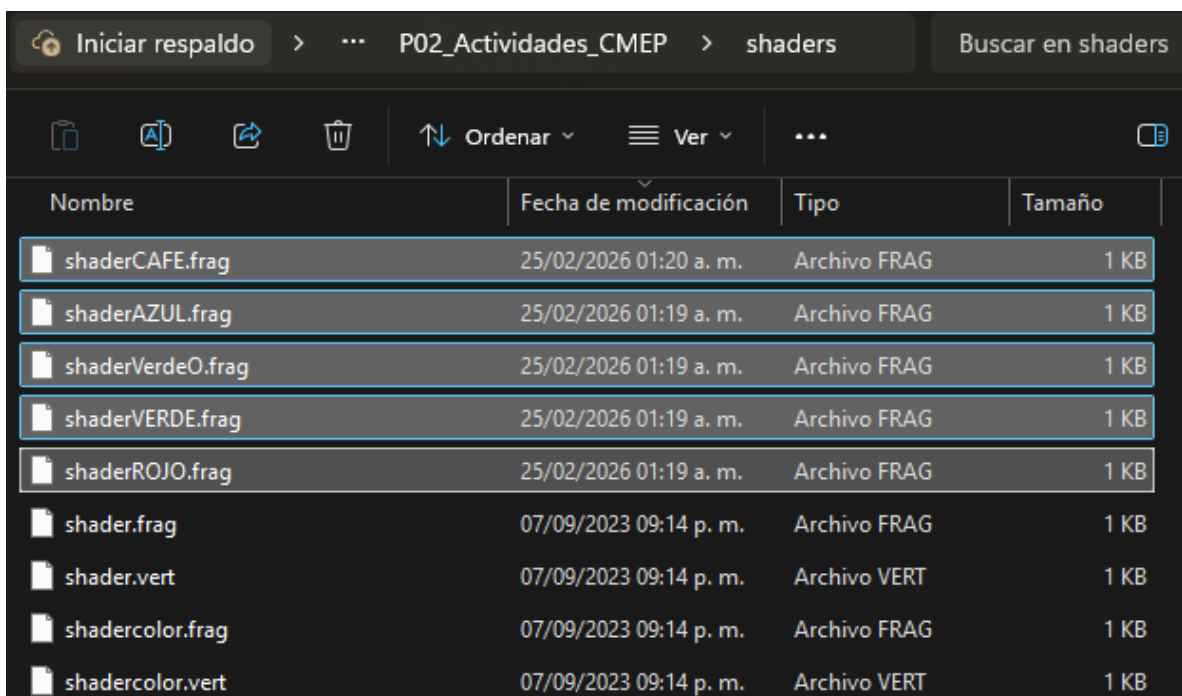
Para la segunda actividad comentamos esta generación de letras y regresamos la instrucción `glm::mat4 projection = glm::perspective(glm::radians(60.0f), 800.0f / 800.0f, 0.1f, 100.0f);` En esta segunda actividad se pidió generar el dibujo de la casa de la clase, pero en lugar de instanciar triángulos y cuadrados tendríamos que instanciar pirámides y cubos, para esto se nos pide crear shaders diferentes de los colores: rojo, verde, azul, café y verde oscuro en lugar de usar el shader con el color clamp.

Comenzamos entonces con la creación de estos shaders, en Visual Studio Code colocamos el mismo código que tenía `shader.frag`, con la diferencia de que aquí asignamos directamente el color que queremos en código RGBA.



```
#version 330
in vec4 vColor;
out vec4 color;
void main() {
    color = vec4(0.620, 0.290, 0.302, 1.0); // RGBA ROJO
}
```

Repetimos este proceso para otros 4 shaders con las variaciones de color necesaria y nombres distintos, obteniendo por resultado los siguientes archivos guardados en la carpeta de shaders.



Nombre	Fecha de modificación	Tipo	Tamaño
shaderCAFE.frag	25/02/2026 01:20 a. m.	Archivo FRAG	1 KB
shaderAZUL.frag	25/02/2026 01:19 a. m.	Archivo FRAG	1 KB
shaderVerdeO.frag	25/02/2026 01:19 a. m.	Archivo FRAG	1 KB
shaderVERDE.frag	25/02/2026 01:19 a. m.	Archivo FRAG	1 KB
shaderROJO.frag	25/02/2026 01:19 a. m.	Archivo FRAG	1 KB
shader.frag	07/09/2023 09:14 p. m.	Archivo FRAG	1 KB
shader.vert	07/09/2023 09:14 p. m.	Archivo VERT	1 KB
shadercolor.frag	07/09/2023 09:14 p. m.	Archivo FRAG	1 KB
shadercolor.vert	07/09/2023 09:14 p. m.	Archivo VERT	1 KB

Ahora, dentro de nuestro código principal, colocamos los shaders recién creados.

```
22      //Vertex Shader
23      static const char* vShader = "shaders/shader.vert";
24      static const char* fShader = "shaders/shader.frag";
25      static const char* vShaderColor = "shaders/shadercolor.vert";
26      static const char* fShaderColor = "shaders/shadercolor.frag";
27      //shaders nuevos se crearían acá (El vShader será el mismo para todos)
28      static const char* fShaderRojo = "shaders/shaderROJO.frag";
29      static const char* fShaderVerde = "shaders/shaderVERDE.frag";
30      static const char* fShaderAzul = "shaders/shaderAZUL.frag";
31      static const char* fShaderCafe = "shaders/shaderCAFE.frag";
32      static const char* fShaderVerdeOscuro = "shaders/shaderVerdeO.frag";
```

Dentro de la función `void CreateShaders()`, instanciamos estos mismos, obteniendo shaders nuevos del índice [2] al índice [6]

```
283      void CreateShaders()
284      {
285          Shader *shader1 = new Shader(); //shader: objetos: cubo y pirámide
286          shader1->CreateFromFiles(vShader, fShader);
287          shaderList.push_back(*shader1);
288          Shader *shader2 = new Shader(); //shader para VAO: letras
289          shader2->CreateFromFiles(vShaderColor, fShaderColor);
290          shaderList.push_back(*shader2);
291
292          // NUEVOS SHADERS PARA LA CASA 3D
293          Shader* shaderRojo = new Shader();
294          shaderRojo->CreateFromFiles(vShader, fShaderRojo);
295          shaderList.push_back(*shaderRojo); // Índice 2 (ROJO)
296
297          Shader* shaderVerde = new Shader();
298          shaderVerde->CreateFromFiles(vShader, fShaderVerde);
299          shaderList.push_back(*shaderVerde); // Índice 3 (VERDE)
300
301          Shader* shaderAzul = new Shader();
302          shaderAzul->CreateFromFiles(vShader, fShaderAzul);
303          shaderList.push_back(*shaderAzul); // Índice 4 (AZUL)
304
305          Shader* shaderCafe = new Shader();
306          shaderCafe->CreateFromFiles(vShader, fShaderCafe);
307          shaderList.push_back(*shaderCafe); // Índice 5 (CAFE)
308
309          Shader* shaderVerdeOscuro = new Shader();
310          shaderVerdeOscuro->CreateFromFiles(vShader, fShaderVerdeOscuro);
311          shaderList.push_back(*shaderVerdeOscuro); // Índice 6 (VERDE OSCURO)
```

Finalmente, dentro de `while (!mainWindow.getShouldClose())` colocamos los cubos y pirámides, haciendo que estos usen los shaders recién creados mediante `shaderList[X].useShader();`

```

371 // -----
372 // DIBUJO DE CASITA 3D -----
373 // -----
374 // 1. BASE DE LA CASA -----
375 shaderList[2].useShader(); // Shader Rojo (Índice 2)
376 uniformModel = shaderList[2].getModelLocation();
377 uniformProjection = shaderList[2].getProjectLocation();
378 // Fachada
379 model = glm::mat4(1.0);
380 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -5.0f));
381 model = glm::scale(model, glm::vec3(1.9f, 2.0f, 1.0f));
382 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
383 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
384 meshList[1]->RenderMesh(); // Cubo
385
386 // 2. TECHO DE LA CASA -----
387 shaderList[4].useShader(); // Shader azul (Índice 4)
388 uniformModel = shaderList[4].getModelLocation();
389 uniformProjection = shaderList[4].getProjectLocation();
390 // Techo
391 model = glm::mat4(1.0);
392 model = glm::translate(model, glm::vec3(0.0f, 1.50f, -5.0f));
393 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
394 model = glm::scale(model, glm::vec3(2.2f, 0.8f, 1.0f));
395 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
396 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
397 meshList[0]->RenderMesh(); // Pirámide
398
399 // 3. VENTANAS Y PUERTA -----
400 shaderList[3].useShader(); // Shader Verde (Índice 3)
401 uniformModel = shaderList[3].getModelLocation();
402 uniformProjection = shaderList[3].getProjectLocation();
403 // Ventana 1
404 model = glm::mat4(1.0);
405 model = glm::translate(model, glm::vec3(0.4f, 0.3f, -4.4f));
406 model = glm::scale(model, glm::vec3(0.55f, 0.55f, 0.2f));
407 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
408 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
409 meshList[1]->RenderMesh(); // Cubo
410 // Ventana 2
411 model = glm::mat4(1.0);
412 model = glm::translate(model, glm::vec3(-0.4f, 0.3f, -4.4f));
413 model = glm::scale(model, glm::vec3(0.55f, 0.55f, 0.2f));
414 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
415 meshList[1]->RenderMesh(); // Cubo
416 // Puerta Central
417 model = glm::mat4(1.0);
418 model = glm::translate(model, glm::vec3(0.0f, -0.55f, -4.4f));
419 model = glm::scale(model, glm::vec3(0.55f, 0.80f, 0.2f));
420 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
421 meshList[1]->RenderMesh(); // Cubo
422
423 // 4. TRONCOS DE LOS ÁRBOLES -----
424 shaderList[5].useShader(); // Shader Café (Índice 5)
425 uniformModel = shaderList[5].getModelLocation();
426 uniformProjection = shaderList[5].getProjectLocation();
427 // Tronco 1
428 model = glm::mat4(1.0);
429 model = glm::translate(model, glm::vec3(-1.68f, -0.695f, -4.8f));
430 model = glm::scale(model, glm::vec3(0.55f, 0.70f, 0.2f));
431 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
432 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
433 meshList[1]->RenderMesh(); // Cubo

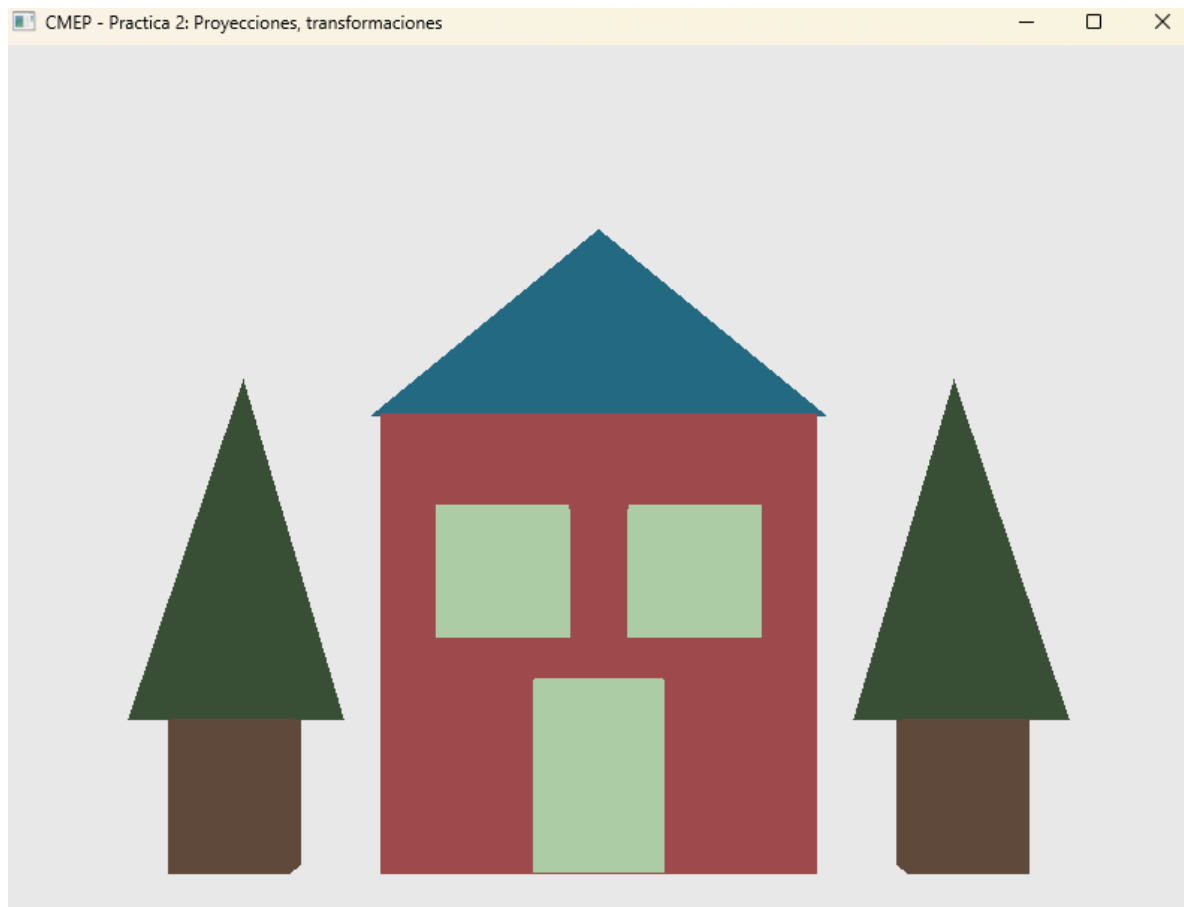
```

```

434 // Tronco 2
435 model = glm::mat4(1.0);
436 model = glm::translate(model, glm::vec3(1.68f, -0.695f, -4.8f));
437 model = glm::scale(model, glm::vec3(0.55f, 0.70f, 0.2f));
438 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
439 meshList[1]->RenderMesh(); // Cubo
440
441 // 5. HOJAS DE LOS ÁRBOLES -----
442 shaderList[6].useShader(); // Shader Verde Oscuro (Índice 6)
443 uniformModel = shaderList[6].getModelLocation();
444 uniformProjection = shaderList[6].getProjectLocation();
445 // Tronco 1
446 model = glm::mat4(1.0);
447 model = glm::translate(model, glm::vec3(1.68f, 0.450f, -4.8f));
448 model = glm::scale(model, glm::vec3(1.0f, 1.6f, 0.4f));
449 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
450 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
451 meshList[0]->RenderMesh(); // Pirámide
452 // Tronco 2
453 model = glm::mat4(1.0);
454 model = glm::translate(model, glm::vec3(-1.68f, 0.450f, -4.8f));
455 model = glm::scale(model, glm::vec3(1.0f, 1.6f, 0.4f));
456 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
457 meshList[0]->RenderMesh(); // Pirámide

```

Ejecución



PROBLEMAS PRESENTADOS

Para esta práctica los únicos problemas presentados fueron el orden de colocación de los shaders, más específicamente en las instrucciones de `shaderList[X].useShader()`; pues se intentó tener una mejor organización con estos y se mandaron todos a una misma parte de código, pero se comprendió que estos son como activadores de los shaders y colocarlos todos juntos haría que solo parezca que se activo el ultimo. Se resolvió solo colocándolos como estaban originalmente, para evitar salidas como la casita coloreada de un solo color.

CONCLUSIONES

Sobre los ejercicios del reporte

La mayor complejidad que se tuvo fue el uso de los shaders, aunque en esencia solo eran códigos pequeños no se comprendía muy bien que hacían y como lo hacían, especialmente si a cambiarlos activamente se refiere. Ambos ejercicios fueron en esencia menos complejos de lo esperado, debido a que ya se contaba con cosas como las coordenadas base para las letras y la disposición de figuras para las partes de la casa.

Comentarios generales

La explicación fue clara, aunque por falta de tiempo se entiende que no se pudiera explicar más sobre los shaders y su vinculación entre los archivos de texto y el código principal.

Cierre

Gracias a esta práctica se consolidaron las bases para el uso de transformaciones geométricas, ya sea en aspectos aparentemente en 2D al usar una proyección ortogonal, o en 3D al usar una proyección en perspectiva. Se logro pasar de una etapa donde no se comprendía casi nada del código base, a una donde se pueden ya identificar de manera satisfactoria varias partes del mismo, así como su manejo y sus archivos de apoyo.

REFERENCIAS

- Shader Tutorial. (s.f.). *Shader basics - Fragment shader*. Recuperado el 24 de febrero de 2026, de <https://shader-tutorial.dev/basics/fragment-shader/>
- de Vries, J. (s.f.). Shaders. LearnOpenGL. Recuperado el 24 de febrero de 2026, de <https://learnopengl.com/Getting-started/Shaders>
- Khronos Group. (s.f.). *Fragment Shader*. OpenGL Wiki. Recuperado el 24 de febrero de 2026, de https://wikis.khronos.org/opengl/Fragment_Shader
- OpenGL-Tutorial. (s.f.). *Tutorial 2: The first triangle*. OpenGL-Tutorial. Recuperado el 24 de febrero de 2026, de <http://www.opengl-tutorial.org/beginners-tutorials/tutorial-2-the-first-triangle/>